Politecnico di Milano

Ingegneria del Software – a.a. 2009/10

Appello del 10 Febbraio 2011

Cognome
Nome
Matricola

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 100 punti nominali. La sufficienza si raggiunge indicativamente con 60 punti.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 20)

Si consideri l'astrazione procedurale **sottraiArray**. L'astrazione ha come parametro due array *x* e *y* di elementi *int*. L'array x non è vuoto, e sia x che y non hanno elementi ripetuti.
L'astrazione restituisce un array di int, contenente gli elementi di x (se esistono) che NON sono presenti in y, in ordine qualunque.

Ad es., se x è [8 2 3 5 1], e y = [2 5 8 7], un array restituito è [1 3].

a- **Si scrivano la pre- e post-condizione del metodo, evidenziando tutti gli elementi significativi**

SOL:

```
/*@pre x!=null && y!=null && x.length>0 &&
        (\forall int i; 0<= i && i< x.length-1; (\forall int j; i<j && j< x.length; x[i]!= x[j])) &&
        (\forall int i; 0<= i && i< y.length-1; (\forall int j; i<j && j< y.length; y[i]!= y[j]));
@*/
/*@post (\forall int i; 0\le i && i< \result.length;
                (\exists j; 0<= j && j< x.length; x[j]==\result[i]) &&
                !(\exists j; 0<= j && j< y.length; y[j]==\result[i])) &&
          (\forall int i; 0<= i && i< x.length;
                !(\exists j; 0<= j && j< y.length; y[j]==x[i])) ==>
                (\exists j; 0<= j && j< x.length; x[i]==\result[j])) &&
        (\forall int i; 0<= i && i< \result.length-1; (\forall int j; i<j && j< \result.length;
                \result[i]!= \result[j]))
@*/
//@\assignable nothing;
```

public static int[] sottraiArray(int[] x, int[]y)

b- **Si identifichino le eccezioni necessarie per rendere totale il metodo e si descrivano le opportune modifiche alla specifica di cui al punto (a).**
SOL

Le eccezioni da lanciare sono NullPointerException, EmptyException e DuplicateException.

La specifica diventa (considerando che \old(pre) == pre, in quanto c'e' \assignable nothing):

```
//@pre true;

//@post …post già vista && …pre gia' vista…


//@signals (NullPointerException e) x==null || y ==null;

//@signals (EmptyException e) x.length==0;

/*@signals (DuplicateException e)
        (\exists int i; 0<= i && i< x.length-1; (\exists int j; i<j && j< x.length; x[i]==x[j])) ||
        (\exists int i; 0<= i && i< y.length-1; (\exists int j; i<j && j< y.length; y[i]==y[j]));
@*/
//@\assignable nothing;
```

Esercizio 2 (punti 60)

Le classi **Punto2D**, **Segmento** e **Triangolo**, rappresentano le note entità geometriche in uno spazio euclideo bidimensionale.

Le specifiche di **Punto2D** e di **Segmento** sono le seguenti:

```
/*@ pure @*/ /* public class Punto2D {
      //@ensures (* costruisce un punto di coordinate cartesiane x,y*);
      public Punto(double x, double y) ;

      //@ensures (*restituisce la distanza euclidea fra this e p*);
      public double distanza (Punto2D p);

      //@ensures (*restituisce la coord. x di this *);
      public double getX();

      //@ensures (*restituisce la coord. x di this *);
      public double getY();
}

/*@ pure @*/ public class Segmento {
      //@ requires f != null && o!=null;
      //@ensures fine()== f && origine()== o;
      public Segmento(Punto2D o, Punto2D f);

      /*@ensures (* restituisce l'estremo iniziale di this *)
      public Punto2D origine();

      /*@ensures (* restituisce l'estremo finale di this *)
      public Punto2D fine()

      /*@ensures \result == origine().distanza(fine());
      public double lunghezza()
}
```

a) Un triangolo è individuato da tre punti non allineati, ossia i vertici. Se i tre punti sono allineati, il triangolo è degenere e non può essere costruito. Si ipotizzi che sia definito un metodo statico (da NON specificare né implementare):

   public static boolean allineati (Punto2d p1, Punto2D p2, Punto2D p3);

   che restituisce true se, e solo se, i 3 punti p1,p2 e p3 sono allineati.

   La base del **triangolo** è definita, ai nostri scopi, convenzionalmente come il lato *maggiore o uguale* agli altri due, il latoMinore è il lato minore o uguale agli altri due, mentre il latoMedio è il rimanente. Il metodo **traslaOrizzontale** incrementa o decrementa le coordinate dei vertici di un valore DeltaX. **Completare negli spazi vuoti la specifica JML della seguente classe Triangolo.**

```
public class Triangolo {
//Tipo mutabile
      //@pre    v1!=null && v2!= null && v3!=null
      /*@post   !allineati(v1,v2,v3)&&
      (* this è un triangolo | {base().origine(), base().fine() ,latoMinore().fine()} = = { v1,v2, v3}*) &&
      base().origine == latoMinore().origine() && base().fine()== latoMedio().origine() &&
      latoMinore().fine()==latoMedio().fine();
      @*/

      //@signals (TriangoloDegenereException e) allineati(v1,v2,v3);
      public Triangolo (Punto2D v1, Punto2D v2, Punto2D v3) throws TriangoloDegenereException;

      //@post   \result.lunghezza() >= latoMedio().lunghezza()
      public /*@ pure @*/ Segmento base ();
```

```
//@post   \result.lunghezza() <= latoMedio().lunghezza()&& ! \result.equals(base());
public /*@ pure @*/  Segmento latoMinore ();

//@post    !\result.equals(base()) && ! \result.equals(lato Minore()=;
public /*@ pure @*/  Segmento latoMedio();

/*@  base().origine().getX()==\old(base().origine().getX())+DeltaX &&
     base().fine().getX()==\old(base().fine().getX())+DeltaX &&
     latoMinore().fine().getX()==\old(latoMinore().fine().getX())+DeltaX &&
     base().origine().getY()==\old(base().origine().getY())&&
     base().fine().getY()==\old(base().fine().getY())&&
     latoMinore().fine().getY()==\old(latoMinore().fine().getY()) &&
     latoMinore().fine()==latoMedio().fine() && latoMedio().origine()==base().fine() &&
     LatoMinore().origine()==base().origine();
@*/
public traslaOrizzontale(double DeltaX);

}
```

NB: Nella postcondizione del costruttore, si ipotizza per semplicità che l'origine della base coincida con l'origine del latoMinore e che la fine della base coincida con l'orgine del lato medio (e che quindi la fine del latoMinore coincida con la fine del latoMedio. Questa ipotesi non viola la specifica data informalmente, in quanto, dati i due punti di cui è costituito un segmento, la scelta di quale sia l'origine e quale sia la fine è arbitraria.
Nelle postcondizioni di lato Minore e di latoMedio, asseriamo anche che i segmenti sono fra loro tutti diversi (altrimenti in caso di triangolo isoscele o equilatero potrebbe essere restituito lo stesso segmento ad esempio come base e latoMedio). In traslaOrizzontale, la postcondizione assicura che tutti i tre vertici del triangolo sono traslati correttamente, essendo certi che i tre vertici del triangolo sono base.origine, base.fine e latoMinore.fine.

b) **Scrivere la proprietà che la base ha lunghezza maggiore o uguale a quella degli altri lati usando un *invariante pubblico.***

```
public invariant base().lunghezza()>=lato Minore().lunghezza() &&
                 base().lunghezza()>=latoMedio().lunghezza();
```

c) Si consideri un'estensione mutabile di Triangolo, detta *TriangoloBaseVariabile*, che rispetto a a Triangolo definisce un nuovo metodo *cambiaBase().*

Il metodo cambia Base riceve come parametro un segmento *nuovaBase* e aggiorna il triangolo adottando come nuova base *nuovaBase*, purche' questa sia maggiore o uguale degli altri lati (e il triangolo risultante non sia degenere).

Specificare il metodo cambiaBase.

```
//@pre      nuovaBase.lunghezza()>=latoMedio().lunghezza();
/*@post (!allineati(nuovaBase.origine(); nuovaBase.fine(); \old(latoMinore().fine())= &&
  (      base().origine() ==nuovaBase.origine() && base().fine()== nuovaBase.fine() ||
         base().fine() ==nuovaBase.origine() && base().origine()== nuovaBase.fine()) &&
  latoMinore().fine() == \old(latoMinore().fine()) &&
  base().origine() == latoMinore().origine() && base().fine()== latoMedio().origine() &&
  latoMinore().fine()==latoMedio().fine();
@*/
/*@signals (TriangoloDegenereException e)
  allineati(nuovaBase.origine(); nuovaBase.fine(); \old(latoMinore().fine())));
@*/
public void cambiaBase(Segmento nuovaBase) throws TriangoloDegenereException;
```

d) TriangoloBaseVariabile verifica il principio di sostituzione di Liskov rispetto a Triangolo? Giustificare la risposta.

SI. L'unico modo in cui l'aggiunta di un metodo può invalidare il principio di sostituzione è tramite la violazione regola delle proprietà. Le proprietà di un oggetto Triangolo sono: 1) rappresenta un triangolo non degenere; 2) la base è maggiore o uguale agli altri due lati; 3) il latoMinore è minore o uguale agli altri due lati.
Le proprietà 1, 2 e 3 sono ancora garantite dal metodo cambiaBase(). Si osservi però che una specifica meno "attenta" violerebbe facilmente la proprietà 3 (è indispensabile che origine fine della base possano essere scambiate, se necessario, in modo da garantre la proprietà che l'origine della base sia anche l'origine del lato minore).

e) Si consideri una classe *TriangoloEquilatero*, definita come un *Triangolo* in cui però tutti i lati sono sempre uguali.

Si dica se il principio di sostituzione di Liskov può essere verificato costruendo TriangoloEquilatero come estensione di Triangolo.

Si, è possibile in quanto l'unico metodo di TriangoloEquilatero che ha una specifica differente da quella corrispondente di Triangolo è il costruttore, che tuttavia non è ereditato. Quindi la regola dei metodi è verificata. La regola delle proprietà è pure verificata, in quanto anche un TriangoloEquilatero è un Triangolo non degenere, la cui base è non minore degli altri lati, e il latoMinore è non maggiore degli altri lati.

Cosa succederebbe se TriangoloEquilatero fosse definita invece come estensione di TriangoloBaseVariabile? Giustificare la risposta.

Non si tratta di violazione del principio di sostituzione, ma di violazione delle proprietà fondamentali di una classe.
infatti, il principio di sostituzione resterebbe verificato, ma una chiamata del metodo cambiaBase potrebbe invalidere la proprietà di TriangoloEquilatero che tutti i lati sono uguali. Pertanto non è possibile definire TriangoloEquilatero come estensione di TriangoloBaseVariabile, in quanto l'oggetto risultante potrebbe evolvere violando la proprietà di essere equilatero.

Esercizio 3 (punti 20)
Sia dato il seguente rep per la classe TriangoloBaseVariabile

private Segmento[3] lati;

Il rep è definito in modo che *lato[0], lato[1]* e *lato[2]* memorizzano, in un ordine qualunque, come *segmenti* i tre lati del triangolo, indipendentemente dall'essere questi la base, il latoMinore o il latoMedio.

Scrivere l'invariante di rappresentazione della classe.

```
private invariant (\forall int i; 0<=i && i<=2; lato[i]!=null)  &&
        (\exists int i;  0<=i && i<=2;
                (\exists int j;  && 0<=j && j<=2;
                        (\exists int k;  0<=i && i<=2;
                                i!=j && i!=k && j!=k &&
                                lato[i].origine() == lato[j].origine() &&
                                lato[i].fine() == lato[k].origine() &&
                                lato[j].fine() == lato[k].fine)) &&
                                !allineati(lato[i].origine(), lato[i].fine(), lato[j].fine()) &&
                                lato[i].lunghezza()>=lato[k].lunghezza() &&
                                lato[k].lunghezza()>= lato[j].lunghezza));
```

NB: i,j,k rappresentano rispettiva,ente gli indici della base, del lato minore e del lato medio.

In base all'invariante e alle specifiche date, implementare il metodo cambiaBase().

```
int indBase;
if (lato[0] .lunghezza() >= lato[1].lunghezza())
        if (lato[0].lunghezza()>= lato[2].lunghezza())
                indBase = 0;
        else indBase=2;
else    if (lato[1].lunghezza()>=lato[2].lunghezza())
                indBase=1;
        else indBase=2;
lato[indBase]= nuovaBase;
lato[(indBase+1) % 3].origine()=nuovaBase.origine();
lato[(indBase+2) % 3].origine()=nuovaBase.fine();
```